Intelligent Robotics Assignment 2

ROS2 — Swap those cubes (with UR-5)!

GitHub Repository Videos Drive

Ferracane Salvatore* Vianello Giacomo†

1 Introduction and Overview

This report presents our solution to *Assignment 2*, which required the development of a complete manipulation pipeline for the UR-5 robotic arm in simulation. The objective was to detect two cubes equipped with AprilTags, estimate their poses with respect to the robot, and execute a robust pick–and–place sequence to swap their positions on the table. In addition to geometric pose estimation, the system was required to classify the color of each cube, reconstruct the surrounding table geometry for collision avoidance, and maintain a consistent virtual scene within MoveIt throughout the manipulation process.

Our architecture follows a *hybrid PLAN, SENSE–ACT* paradigm (see Fig. 1). In this formulation, the sensing layer extracts both geometric and semantic information from the environment, the planning layer supervises the manipulation workflow and coordinates subsystem interactions, and the acting layer executes motion commands through MoveIt 2 while ensuring collision safety. This paradigm is particularly suitable for our scenario: once the cube poses are estimated, the robot must autonomously compute safe grasping and placement configurations, manage object attachments within the planning scene, and perform precise Cartesian and joint-space motions to complete the swap.

In summary, this document provides a structured overview of the system architecture and its main components. A detailed description of each node, along with the full implementation, is available in the official GitHub repository, while demonstration videos can be found in the accompanying media folder.

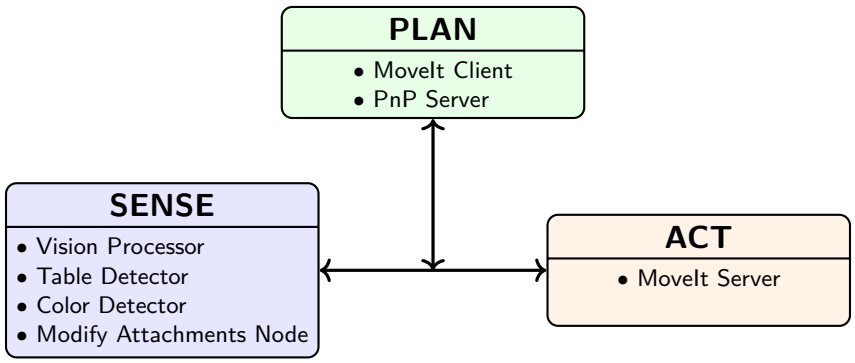


Figure 1: Hybrid architecture: the Plan layer supervises the reactive Sense–Act loop. Each block contains the ROS 2 nodes developed to achieve the final navigation pipeline.

*salvatore.ferracane@studenti.unipd.it
†giacomo.vianello.2@studenti.unipd.it

1.1 SENSE

The *SENSE* layer provides all perceptual and scene–maintenance capabilities required for manipulation. It integrates geometric, semantic, and structural information from the environment and ensures that the MoveIt planning scene remains consistent throughout the task.

The **Vision Processor** transforms AprilTag detections into the `base_link` frame, enabling accurate localization of both cubes. The **Table Detector** reconstructs the two supporting tables through HSV segmentation and 3D projection, publishing them as `CollisionObjects` for collision avoidance. The **Color Detector** classifies cube colors using TF-guided ROI projection and HSV filtering. Finally, the **Modify Attachments** node updates the MoveIt planning scene by converting objects between world and attached representations, ensuring that the virtual environment always reflects the robot's current state.

Together, these nodes form a coherent perceptual layer that supplies the planning and acting components with reliable and up-to-date information (see Fig. 2)

SENSE

- **Vision Processor**: transforms AprilTag poses into `base_link`.
- **Table Detector**: segments and publishes tables as MoveIt `CollisionObjects`.
- **Color Detector**: performs ROI-based HSV segmentation for cube color classification.
- **Modify Attachments**: updates the MoveIt scene by attaching/detaching objects.

Figure 2: SENSE block: high-level nodes functionality.

1.2 PLAN

The *PLAN* layer oversees the complete manipulation pipeline and manages the coordination between perception and execution. It consists of two nodes—the MoveIt Client and the PNP Server—which together implement a unified planning layer: the high-level FSM governs the task logic, while the PnP Server handles the low-level manipulation primitives. This division enables a clear, modular, and extensible supervisory architecture (see Fig. 3).

The **MoveIt Client** node acts as the high-level Finite State Machine (FSM) that governs the entire task progression: it detects both cubes, requests the PnP Server to swap the first cube and then the second, and finally returns the arm to the HOME configuration once the sequence is complete. The structure is intentionally extensible, allowing future tasks—such as adding a third cube—to be integrated with minimal changes. The MoveIt Client queries the perception layer for cube poses, computes consistent end-effector orientations, triggers the pick-and-place routine, and supervises the overall workflow. All operations are executed asynchronously to ensure responsiveness and avoid blocking behavior.

Complementing this, the **PnP Server** encapsulates the full pick-and-place routine inside its own dedicated FSM (see Fig. 4). It coordinates arm and gripper motions, executes both Cartesian and joint-space trajectories, and interacts with the attachment manager to keep the MoveIt planning scene consistent during grasp and release events.

PLAN

- **MoveIt Client**: high-level orchestrator implementing the FSM for the full manipulation workflow.
- **PnP Server**: executes the complete pick–and–place sequence through a dedicated ROS 2 action.

Figure 3: PLAN block: high-level nodes functionality.

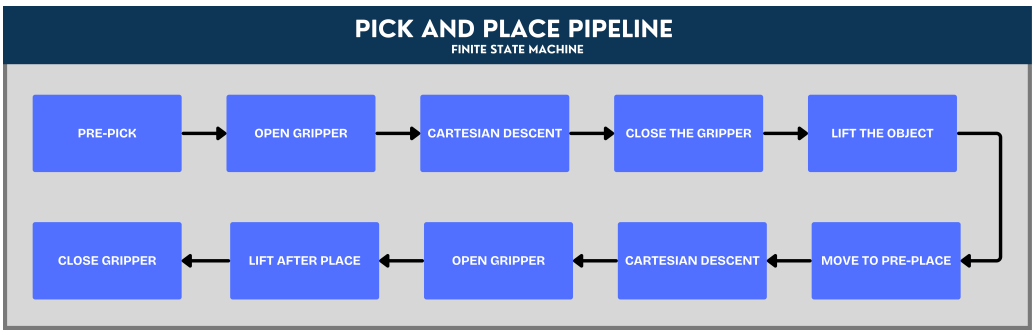


Figure 4: Pick and Place Pipeline.

1.3 ACT

The *ACT* layer is responsible for executing all robot motions required during the manipulation task. It provides the physical interface between the high-level planning logic and the UR-5 hardware abstraction, ensuring that trajectories are executed safely, accurately, and in full consistency with the planning scene (see Fig. 5).

The core component of this layer is the **MoveIt Server**, which exposes both the `/move_robot` action and the `/plan_gripper` service. Through these interfaces, the server handles Cartesian paths, joint-space motions, precise pose-based point-to-point (PTP) movements, and gripper actuation. All trajectories are computed using MoveIt 2's planning pipelines, which guarantee collision-aware execution based on the scene provided by the SENSE layer.

Since the maintenance of the planning scene is handled entirely within the SENSE layer, the ACT layer focuses exclusively on motion execution based on the current planning scene. It translates high-level manipulation commands into precise and collision-safe robot motions, forming the final stage of the *hybrid PLAN, SENSE–ACT* pipeline.

ACT

- **MoveIt Server**: executes Cartesian, joint-space, and PTP motions; controls the Robotiq gripper.

Figure 5: ACT block: high-level nodes functionality.

2 UML: A (quite) Deeper Explaination

Figure 6 provides a unified overview of the ROS 2 nodes developed for this assignment and illustrates the flow of information across topics, services, and actions. The diagram highlights the modular structure of the system and clarifies how perception, planning, and execution interact to achieve the final manipulation task.

The execution pipeline begins with the **Vision Processor**, which continuously monitors the TF tree populated by `apriltag_ros`. Whenever the `/get_cube_pose` service is invoked, the node retrieves the transform between `base_link` and the requested AprilTag frame, returning the cube pose to the planning layer. In parallel, the **Table Detector** processes the RGB camera stream to identify the two tables, projects their centroids into 3D, and publishes them as `CollisionObjects` to MoveIt. The **Color Detector** complements this information by classifying the color of each cube through TF-guided ROI extraction and HSV segmentation. Finally, the **Modify Attachments** node maintains the consistency of the MoveIt planning scene by attaching or detaching objects when requested by the manipulation layer.

Once the perceptual information is available, the **MoveIt Client** initiates its finite-state machine. It first queries the Vision Processor to obtain the poses of both cubes, then computes the corresponding pick and place poses by enforcing a fixed end-effector orientation and applying small offsets for grasping and motion planning stability. When the poses are ready, the client triggers the

PnP Server through the `/pick_place` action. This action encapsulates the entire manipulation sequence: approaching the cube, opening the gripper, performing a Cartesian descent, grasping, attaching the object, lifting, moving to the placement location, releasing the cube, and detaching it from the gripper (Fig.4).

During this process, the PnP Server interacts with two key interfaces exposed by our **MoveIt Server**: the `/move_robot` action, used for Cartesian, joint-space, and pose-based PTP motions; and the `/plan_gripper` service, used to open or close the Robotiq gripper. These interactions ensure that all motions are collision-aware and consistent with the planning scene maintained by the SENSE layer.

After both cubes have been successfully swapped, the MoveIt Client sends a final joint-space goal to return the robot to the HOME configuration, concluding the manipulation pipeline and printing the color of the cubes on the terminal. For a clearer understanding of the full workflow, including a detailed description of each node, we suggest the reader to refer to the accompanying demonstration video and the project's GitHub repository.

3 Conclusion

As demonstrated throughout this report and in the video, the proposed architecture successfully fulfills all requirements of Assignment 2.

The system integrates perception, planning, and execution into a coherent *hybrid PLAN, SENSE–ACT* pipeline capable of detecting the cubes, estimating their poses, classifying their colors, reconstructing the surrounding table geometry, and executing a complete pick–and–place sequence to swap their positions using the UR-5 manipulator.

The perceptual layer provides a reliable and up-to-date representation of the environment, combining AprilTag-based pose estimation, table reconstruction, color classification, and scene maintenance through attachment management. The planning layer abstracts the manipulation logic into a clean supervisory structure, coordinating the task through a finite-state machine and encapsulating the full pick–and–place routine into a dedicated ROS 2 action. The execution layer ensures precise and collision-aware motion control through MoveIt 2, translating high-level commands into feasible trajectories.

However, it is worth noting that the ACT layer may occasionally **fail** to execute a requested motion. This is due to the stochastic nature of the sampling-based planners used by MoveIt (e.g., OMPL planners such as RRTConnect), which do not guarantee a solution at every attempt. In such cases, the planner may be unable to find a valid trajectory despite the existence of one, leading to sporadic motion failures during Cartesian or joint-space planning. These events are inherent to the probabilistic nature of the underlying algorithms and do not reflect a flaw in the system architecture, but rather a limitation of the planning framework itself.

Overall, the system demonstrates a robust and modular design, where each component contributes to a well-defined stage of the manipulation pipeline. Future extensions may include deterministic fallback planners, retry strategies, or hybrid planning approaches to further mitigate stochastic failures and enhance reliability.

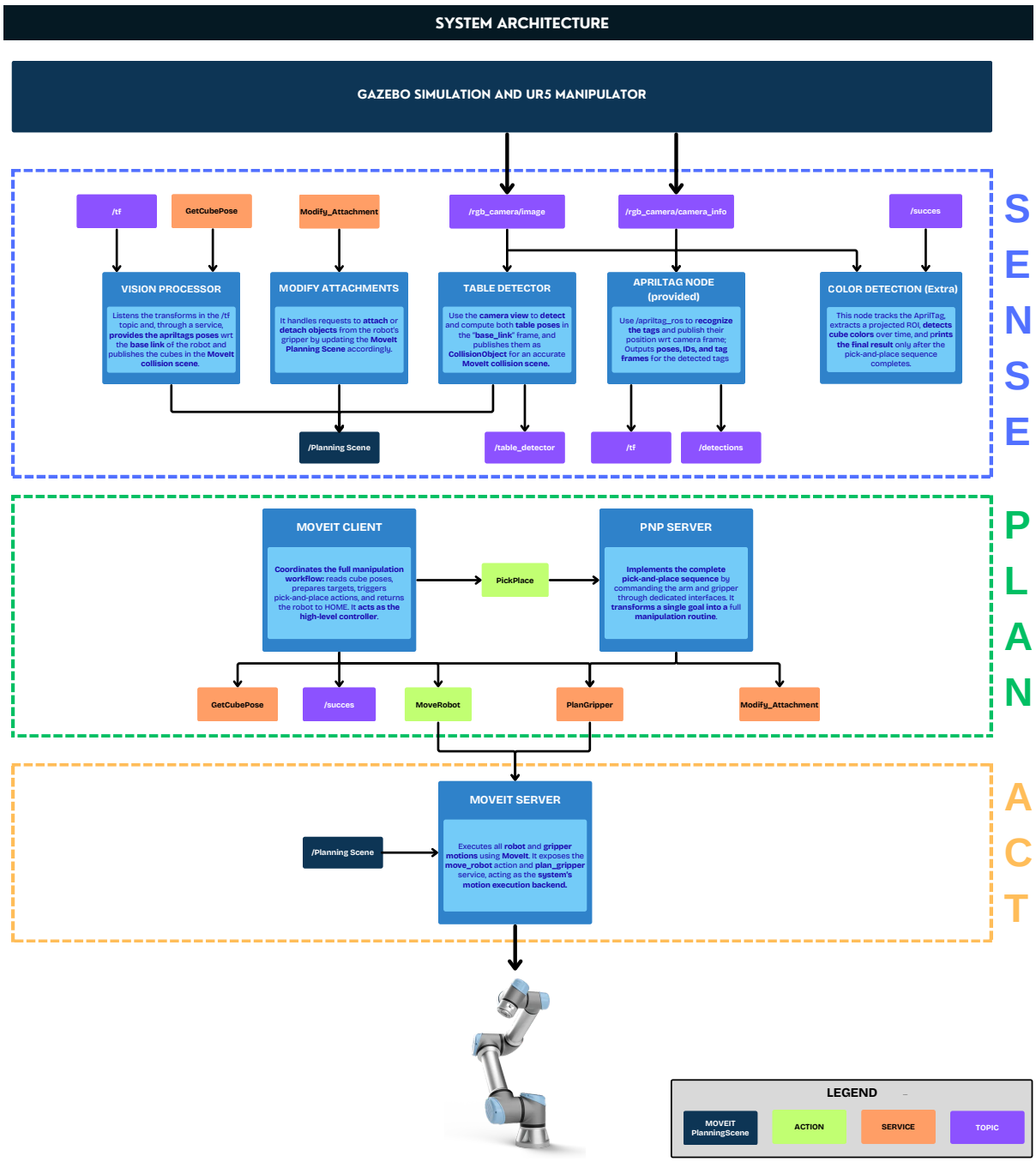


Figure 6: Unified Modeling Language (UML) diagram of the project.